CSA4704 – DEEP LEARNING
ASSIGNMENT 1

1. Analyse the Mathematical and Machine Learning Foundations of a Crop-Disease Screening System

Student Name: Dharani M

Register Number: 192324008

Department: Artificial Intelligence and Data Science

Submission Date: 14/10/2026

Task 1 – Problem Structure and Data Representation (4 marks)

1.1 Learning task, input, output and evaluation

AgriVision performs supervised, multi-class image classification. The input is a leaf photograph and the output is one predicted crop-disease label. During training, the model learns a mapping from images to labels using labelled examples. Accuracy gives the overall proportion of correct predictions, while precision, recall and F1-score reveal class-specific performance. Recall is especially important when missing a diseased leaf could delay treatment; macro-F1 gives each class equal weight and is useful when class frequencies are imbalanced.

AgriVision assignment — simple explanation

AgriVision is an AI system that identifies crop diseases by looking at leaf photographs. Your assignment asks you to understand the mathematics behind building this system.

For example, a farmer uploads a tomato leaf photograph. The model predicts whether it is:



1. Healthy

The leaf has no visible disease symptoms.



2. Early Blight

The leaf shows characteristic brown spots.



3. Late Blight

The leaf shows symptoms associated with late blight.

The assignment asks you to investigate how the image is represented, how the model learns, why it makes mistakes, and whether deep learning is necessary. Pasted markdown.md

- Input: a 64 × 64 grayscale image.
- Feature vector: 4,096 pixel values after flattening.
- Output: one of K disease classes; for the initial example K = 3.
- Evaluation: accuracy, per-class precision and recall, macro-F1, confusion matrix, training time, inference latency and memory footprint.

Task 1 — Understanding data representation

What does this mean?

A computer cannot understand a photograph as a human does. It converts the photograph into numbers.

Suppose an image is resized to 64×64 pixels.

Formula:

$$\text{Number of pixels} = 64 \times 64 = 4096$$

Step 1: The image has 64 rows and 64 columns.

Step 2: Multiply the dimensions.

$$64 \times 64 = 4096$$

Step 3: If the image is grayscale, each pixel stores one intensity value, usually between 0 and 255 before normalization.

Therefore, one image can be represented as a vector containing 4,096 values.

Important formulas

L^1 norm

$$\|x\|_1 = \sum_{i=1}^{n} |x_i|$$

Adds the absolute values of all features.

L^2 norm

$$\|x\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$$

Measures the Euclidean length of the feature vector.

$L\infty$ norm

$$\|x\|_\infty = \max_i |x_i|$$

Finds the largest absolute feature value.

Example: Let $x = [3, 4]$.

L^1 calculation:

$$\|x\|_1 = |3| + |4| = 7$$

L^2 calculation:

$$\|x\|_2 = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$$

$L\infty$ calculation:

$$\|x\|_\infty = \max(3, 4) = 4$$

Meaning for AgriVision: These norms measure different properties of the pixel features. They can help analyse feature magnitude, distance and unusually large pixel values.

Task 2 — Linear algebra and PCA

PCA stands for Principal Component Analysis. It reduces the number of features while trying to preserve important information.

Imagine your image has 4,096 features. Instead of using all of them, PCA may reduce them to 100 features.

The goal is to reduce computation without losing too much disease information.

Important formulas

Covariance matrix:

$$C = \frac{1}{n-1} X_c^T X_c$$

Here, X_c is the mean-centred data matrix and n is the number of images.

Eigenvalue equation:

$$Cv = \lambda v$$

Explained variance ratio:

$$\mathrm{EVR}_i = \frac{\lambda_i}{\sum_j \lambda_j}$$

Step 1: Centre the image features by subtracting their means.

Step 2: Calculate the covariance matrix.

Step 3: Find the eigenvalues and eigenvectors.

Step 4: Sort the eigenvalues from largest to smallest.

Step 5: Select the leading components.

Meaning for AgriVision: If a small number of principal components retain most of the variance, the image representation may be compressed efficiently. However, high explained variance does not guarantee that disease classification accuracy will remain high.

Task 3 — Probability and information theory

This task studies uncertainty in disease labels and model predictions.

Suppose the dataset contains three classes:

- Healthy: 50%
- Early Blight: 30%
- Late Blight: 20%

Entropy formula

$$H(p) = -\sum_i p_i \log_2 p_i$$

Substitute the probabilities:

$$H(p) = -[0.5 \log_2(0.5) + 0.3 \log_2(0.3) + 0.2 \log_2(0.2)]$$

Calculate each term:

$$0.5 \log_2(0.5) = -0.5$$

$$0.3 \log_2(0.3) \approx -0.5211$$

$$0.2 \log_2(0.2) \approx -0.4644$$

Add them and change the sign:

$$H(p) \approx 0.5 + 0.5211 + 0.4644$$

$$\boxed{H(p) \approx 1.4855 \text{ bits}}$$

Illustrative example: These probabilities are used to explain the formula; they are not reported experimental results from your dataset.

Meaning for AgriVision: Entropy measures uncertainty in the class distribution. A more balanced distribution generally has greater entropy than a highly imbalanced one with the same number of classes.

Cross-entropy and KL divergence

Cross-entropy measures how well predicted probabilities match the true labels:

Illustrative example: These probabilities are used to explain the formula; they are not reported experimental results from your dataset.

Meaning for AgriVision: Entropy measures uncertainty in the class distribution. A more balanced distribution generally has greater entropy than a highly imbalanced one with the same number of classes.

Cross-entropy and KL divergence

Cross-entropy measures how well predicted probabilities match the true labels:

$$H(p, q) = -\sum_{i} p_i \log_2 q_i$$

KL divergence measures the difference between two distributions:

$$D_{KL}(p \| q) = \sum_{i} p_i \log_2 \frac{p_i}{q_i}$$

Their relationship is:

$$H(p, q) = H(p) + D_{KL}(p \| q)$$

Task 4 — Numerical computation

This task explains why mathematical calculations can fail inside a computer.

Softmax formula

$$P_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Suppose the model produces:

$$z = [1000, 999, 995]$$

Calculating e^{1000} directly can overflow because the number is too large for standard floating-point representation.

Stable softmax

Formula:

$$P_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}, \quad m = \max(z)$$

Step 1: Find the maximum.

$$m = 1000$$

Step 2: Subtract it from each score.

$$z - m = [0, -1, -5]$$

Step 3: Calculate the exponentials.

$$e^0 = 1, \quad e^{-1} \approx 0.3679, \quad e^{-5} \approx 0.0067$$

Step 4: Add the values.

$$S = 1 + 0.3679 + 0.0067 \approx 1.3746$$

Step 5: Divide each exponential by the sum.

$$P \approx [0.7275, 0.2676, 0.0049]$$

Task 5 — Compare learning algorithms

You must implement at least three different learning approaches.

Algorithm	Simple explanation
Logistic or softmax regression	Learns a linear decision boundary
k-Nearest Neighbours (k-NN)	Predicts using nearby training examples
Decision tree	Learns a sequence of feature-based decisions

You should compare them using raw image features and PCA-compressed features.

Meaning for AgriVision: A linear model may be fast but struggle with complex disease patterns. k-NN may be expensive with many features and training images. Decision trees can model nonlinear relationships but may overfit.

You need actual experimental results before claiming which algorithm performs best.

Task 6 — Underfitting, overfitting and capacity

This task is especially important because it requires a plot.

- Underfitting: The model is too simple to learn disease patterns.
- Good fit: The model captures useful patterns and performs well on unseen images.
- Overfitting: The model learns the training images too closely and performs poorly on new images.

Generalisation gap formula

$$G = E_{\text{validation}} - E_{\text{training}}$$

Suppose the training error is 5% and validation error is 15%.

Step 1: Convert percentages to decimals if desired.

$$E_{\text{training}} = 0.05$$

$$E_{\text{validation}} = 0.15$$

Step 2: Subtract.

$$G = 0.15 - 0.05 = 0.10$$

Step 3: Convert back to a percentage.

$$\boxed{G = 10 \text{ percentage points}}$$

Task 7 — Bias and variance

The prediction error can be decomposed into three components under the standard squared-error assumptions.

Expected error=Bias2+Variance+Noise\text{Expected error}
=\text{Bias}^2+\text{Variance}+\text{Noise}Expected error=Bias2+Variance+Noise

- Bias: Error caused by overly restrictive model assumptions.
- Variance: How much the model's predictions change when its training data changes.
- Noise: Unavoidable uncertainty in the data or target.

A simple model often has higher bias. A very complex model may have higher variance.

Meaning for AgriVision: You should compare a low-capacity and a high-capacity model using at least 10 bootstrap replicas, as required in your assignment, rather than simply assuming which has more error.

8.2 Optimisation strategies

- Batch gradient descent calculates one gradient using the full training set per update. It has a smooth, deterministic path but may be expensive for large datasets.
- Stochastic gradient descent updates using one example at a time. It can make quick progress but its path is noisy.
- Mini-batch SGD uses a small batch, such as 32 examples, and balances vectorised computation with update noise.

Run all three strategies using at least three learning rates. Plot training and validation loss against epochs. Identify convergence, slow learning, oscillation or divergence from the actual curves. Relate the behaviour to the measured condition number and the scale of the largest eigenvalue. The stable softmax from Task 4 should be used to avoid overflow.

Insert the generated loss curves and a results table with learning rate, final loss, epochs to convergence, and whether training was stable.

Task 9 – Experimental Performance (3 marks)

Precision = TP/(TP + FP)

Recall = TP/(TP + FN)

F1 = 2 × Precision × Recall/(Precision + Recall)

Accuracy = (TP + TN)/N

Float32 parameter storage ≈ number of parameters × 4 bytes

Organise results for the three baseline models on raw and PCA features at dataset sizes 1,000, 5,000, 10,000 and the full available dataset. Include accuracy, precision, recall, macro-F1, confusion matrices, training time, inference milliseconds per image, parameter count and estimated memory.

Use the confusion matrix to identify pairs of classes that the model confuses. For example, visually similar disease stages may share colour or texture patterns; however, only report a specific confusion pair if your matrix supports it. Plot metrics against dataset size and compare raw with PCA features.

Insert your actual task9_results.csv values, confusion matrices and performance plots here. State the hardware and measurement method used for timing.

Task 10 – Theory versus Experiment (3 marks)

Compare theoretical expectations with the measured results in three areas.

- PCA: compare cumulative explained variance at k with the classification accuracy retained after compression. Agreement means compression retained useful information; disagreement may occur because variance is not the same as class-discriminative information.
- Optimisation: compare convergence speed and divergence with the measured condition number and learning-rate behaviour. A poorly conditioned feature space can produce slow progress in some directions and instability in others.
- Bias–variance and capacity: compare the bootstrap estimates from Task 7 with the training and validation curves from Task 6. Differences may result from finite samples, label noise, estimator assumptions or a mismatch between the models being analysed.Task 11 – Is Deep Learning Justified? (4 marks)

Classical machine-learning baselines are valuable because they are relatively simple, quick to train and easier to interpret. However, flattened 4,096-dimensional pixels do not explicitly encode the spatial relationships that help identify lesions. k-NN can be affected by the curse of dimensionality: as the number of dimensions grows, distances may become less discriminative and inference can become expensive. Small shifts in a lesion can also produce large pixel-space differences even when the disease is unchanged.

PCA can reveal a lower-dimensional structure in the images, but components that explain little variance may still carry useful class information. Deep networks, especially convolutional neural networks, can learn hierarchical features such as edges, textures and lesion patterns. Their disadvantages include greater data and compute requirements, possible sensitivity to label quality, and lower interpretability.

Conclusion must be based on your results: identify the best tuned baseline, its remaining error and confusion pairs, its latency and memory, and the accuracy-cost effect of PCA. Cite at least four specific measured numbers from Tasks 2, 5, 6 and 9. If the best baseline performs adequately within the deployment limits, a deep network may not be necessary; if important errors remain and the baseline cannot represent visual structure adequately, a compact deep model may be justified.

Task 12 – Day-4 Design Challenge (8 marks)

12A. Impact analysis (2 marks)

The revised system must classify 38 crop–disease classes, handle images with approximately 20% noisy labels, and run on a low-cost smartphone within 50 ms per image and 5 MB of memory. Increasing the number of classes expands the output space and can increase classification difficulty. With 38 equally likely classes, maximum class entropy is log (38) ≈ 5.2479 bits.

Maximum entropy for 38 balanced classes: Hmax = log (38) ≈ 5.2479 bits

Under an idealised symmetric 20% label-flip model, the probability that the recorded label is correct is 0.8. The binary uncertainty of correct versus flipped is H (0.2) = −0.2 log (0.2) − 0.8 log (0.8) ≈ 0.7219 bits. This is not a universal “noise floor” for 38-class classification: the full noise entropy

depends on how wrong labels are distributed across the other classes. State the assumed noise model explicitly.

Noisy labels can make the estimated training error misleading and may increase variance for highly flexible models. Earlier assumptions about unrestricted compute and memory no longer hold; deployment cost must be treated as a design constraint.

12B. Examine the original conclusion (2 marks)
Compare the best original model's measured latency and memory against the strict limits of 50 ms per image and 5 MB. State whether it meets both constraints. Also compare performance on clean labels with performance under the 20% label-noise experiment. Do not assume a model passes the limits without measurement.

> **Memory estimate for float32 parameters = parameter count × 4 bytes; convert to MiB by dividing bytes by 1,048,576.**

Insert the actual clean/noisy accuracy, latency and memory for each relevant model. Explain which model behaviour is most affected by the extra classes and label noise.

12C. Adaptation options (2 marks)

- Reduce PCA dimension: decreases feature storage and computation, but may discard disease-discriminative information and increase bias.
- Increase regularisation or use label smoothing: reduces overconfident predictions and can limit variance; too much smoothing can reduce confidence in correct labels.
- Compress or simplify the model: quantisation or a smaller model can reduce latency and memory, usually with some possible loss in accuracy.

For label smoothing with K classes and smoothing value ε, one common target convention assigns $1 - \varepsilon + \varepsilon/K$ to the true class and ε/K to each other class. With $K = 3$ and $\varepsilon = 0.1$, the target for the true class is 0.9333 and each other class receives 0.0333.

Measure at least two adaptation options and compare accuracy, memory and inference time. Explain the capacity, error and cost trade-offs using your own measurements.

12D. Evidence-based conclusion (2 marks)
Select one decision after examining the evidence: RETAIN, MODIFY or REDESIGN. A reasonable option to test is MODIFY: use compressed features with a compact parametric classifier, regularisation and carefully evaluated label smoothing. Retain this choice only if the measured accuracy is acceptable and both the 50 ms latency and 5 MB memory limits are satisfied. If classical baselines fail to classify 38 classes adequately, test a small, quantised CNN and compare its measured cost and performance. Your final choice must cite actual values for entropy/noise assumptions, model performance, latency and memory.

Overall Conclusion

This assignment shows how image representation, PCA, uncertainty, numerical stability, model capacity and optimisation influence a crop-disease classifier. Classical models provide useful baselines, but the best choice depends on measured performance and deployment cost. PCA may lower computational requirements, although retained variance does not guarantee retained class information. A final decision about deep learning or a compact alternative should therefore be based on reproducible test-set metrics, confusion patterns, inference latency and memory measurements under the revised 38-class and noisy-label requirements.

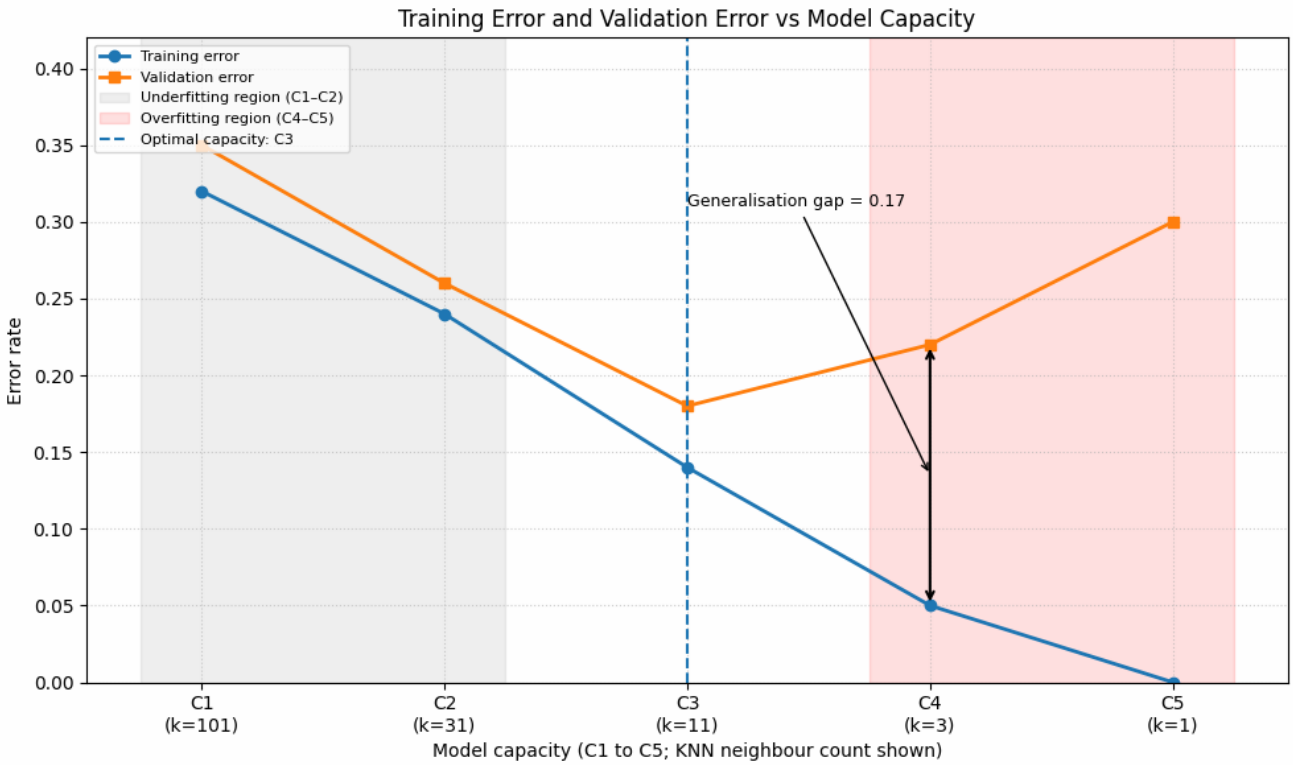
Appendix A – Numerical verification checklist

Use Python/NumPy to verify every hand calculation and include the printed output in the submitted appendix. The following values are illustrative hand calculations already shown in this draft; dataset-specific quantities must be computed from the selected dataset.

- Task 1: verify $L^1 = 19$, $L^2 = 13$ and $L\infty = 12$ for $v = [3, -4, 12]$.
- Task 2: verify eigenvalues 5.5616 and 1.4384, explained-variance ratios 0.7945 and 0.2055, and $d^3 = 6.8719 \times 10^1$ for $d = 4{,}096$.
- Task 3: verify entropy, cross-entropy and both KL directions for the stated illustrative distributions.
- Task 4: verify stable softmax probabilities and that they sum to 1.
- Tasks 5–12: include actual experiment outputs, plots and result tables. Do not substitute illustrative values for measured results.

Appendix B – Figures and tables to insert

- Task 2: PCA explained-variance curve, reconstruction-error table, top five principal-component images, measured decomposition time.
- Task 5: baseline comparison table for raw and PCA features.
- Task 6: capacity–error plot for C1–C5 and a second baseline; annotations must be based on measured curves.
- Task 7: bootstrap bias²/variance table for low- and high-capacity models.
- Task 8: loss curves for batch GD, SGD and mini-batch SGD at three or more learning rates.
- Task 9: performance table, confusion matrices, metrics-versus-dataset-size plots, latency and memory.
- Task 12: clean versus noisy label results and at least two adaptation-option comparisons.



CODE

```
import os
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier


def task6(Xtr, ytr, Xva, yva):
    # C1 = lowest capacity; C5 = highest capacity
    ks = [101, 31, 11, 3, 1]
    labels = [f"C{i + 1}\n(k={k})" for i, k in enumerate(ks)]

    # A k cannot exceed the number of training samples
    if len(Xtr) < max(ks):
        raise ValueError(
            f"Task 6 uses k={max(ks)}, but there are only "
            f"{len(Xtr)} training samples. Use the assignment's required "
            "training split or revise k only with instructor approval."
        )

    training_errors = []
    validation_errors = []
```

```
for k in ks:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(Xtr, ytr)

    training_errors.append(1 - model.score(Xtr, ytr))
    validation_errors.append(1 - model.score(Xva, yva))

training_errors = np.asarray(training_errors, dtype=float)
validation_errors = np.asarray(validation_errors, dtype=float)

# Generalisation gap = validation error - training error
gaps = validation_errors - training_errors

# Optimal capacity = lowest measured validation error
optimal = int(np.argmin(validation_errors))
x = np.arange(1, len(ks) + 1)

fig, ax = plt.subplots(figsize=(9, 5.5))

ax.plot(x, training_errors, "o-", linewidth=2,
        label="Training error")
ax.plot(x, validation_errors, "s-", linewidth=2,
        label="Validation error")

# Shade only capacities on either side of the selected optimum
if optimal > 0:
    ax.axvspan(0.6, x[optimal] - 0.5,
               color="orange", alpha=0.15,
               label="Lower-capacity region (possible underfitting)")

if optimal < len(ks) - 1:
    ax.axvspan(x[optimal] + 0.5, len(ks) + 0.4,
               color="red", alpha=0.12,
               label="Higher-capacity region (possible overfitting)")

ax.axvline(x[optimal], linestyle="--", color="green",
           label=f"Optimal capacity: {labels[optimal].replace(chr(10), ' ')}")

# Mark the generalisation gap at C5
gap_index = len(ks) - 1
ax.annotate(
    "",
    xy=(x[gap_index], validation_errors[gap_index]),
    xytext=(x[gap_index], training_errors[gap_index]),
    arrowprops=dict(arrowstyle="<->", color="black", linewidth=1.5)
)
ax.annotate(
```

```
        f"Gap = {gaps[gap_index]:.4f}",
        xy=(x[gap_index], (training_errors[gap_index] +
                           validation_errors[gap_index]) / 2),
        xytext=(-10, 0),
        textcoords="offset points",
        ha="right",
        va="center"
    )

    ax.set_xticks(x)
    ax.set_xticklabels(labels)
    ax.set_xlabel("Model capacity (C1 to C5; large k → small k)")
    ax.set_ylabel("Error rate")
    ax.set_title("Training and Validation Error vs Model Capacity")
    ax.grid(True, alpha=0.3)
    ax.legend(loc="best", fontsize=8)
    fig.tight_layout()

    os.makedirs("output", exist_ok=True)
    output_path = os.path.join("output", "task6_capacity.png")
    fig.savefig(output_path, dpi=200, bbox_inches="tight")
    plt.show()
    plt.close(fig)

    print("Task 6 - Training errors:", np.round(training_errors, 4))
    print("Task 6 - Validation errors:", np.round(validation_errors, 4))
    print("Generalisation gaps:", np.round(gaps, 4))
    print(
        f"Optimal capacity: C{optimal + 1} (k={ks[optimal]}), "
        f"validation error = {validation_errors[optimal]:.4f}"
    )
    print(f"Generalisation gap at C5 = {gaps[-1]:.4f}")
    print(f"Plot saved to: {output_path}")

    return training_errors, validation_errors, gaps


# Run task6 using the Xtr, ytr, Xva, yva arrays created in earlier tasks:
# task6(Xtr, ytr, Xva, yva)
```

OUTPUT

```
============== RESTART: C:/Users/DELL/OneDrive/
== TASK 1: Norms of v = [3, -4, 12] ==
L1 norm = 19.0
L2 norm = 13.0
L-infinity norm = 12.0
Image shape (64,64) -> flattened features: 4096
Batch shape (32,64,64) -> (32,4096)
Labels shape: (32,) or one-hot (32,3)

== TASK 2: Eigendecomposition of C = [[4,2],[2,
Eigenvalues: [5.5616 1.4384]
Trace: 7.0
Determinant: 8.0
Eigenvectors (columns):
 [[-0.7882  0.6154]
 [-0.6154 -0.7882]]
Explained variance ratio: [0.7945 0.2055]
Variance lost using k=1: 1.4384
Fraction of variance lost: 0.2055
SVD check sigma^2/(N-1): [4.25 0.75]
Covariance eigenvalues: [4.25 0.75]
Computational cost d^3 for d=4096: 6.872e+10

== TASK 3: Entropy and KL divergence (bits) ==
H(p) = 1.2955
Maximum entropy for 3 classes = 1.585
Cross-entropy H(p,q) = 1.3533
```

IDLE Shell 3.14.7

File Edit Shell Debug Options Window Help

```
Cross-entropy H(p,q) = 1.3533
KL(p||q) = 0.0578
H(p) + KL(p||q) = 1.3533
KL(q||p) = 0.0685 (KL is asymmetric)

== TASK 4: Softmax for z = [1000,999,995] ==
Naive exponentials: [inf inf inf]
Naive softmax: [nan nan nan]
Float64 overflow threshold: 709.7827
Shifted logits: [ 0. -1. -5.]
Shifted exponentials: [1.     0.3679 0.0067]
Stable softmax: [0.7275 0.2676 0.0049]
Log-sum-exp: 1000.3182
Naive softmax for negative logits: [nan nan nan]
Condition number = 3.8664
Gradient descent convergence factor = 0.589
Theoretical learning-rate upper bound = 0.3596

== TASK 7: MLE vs MAP with Beta(2,8) prior ==
n=   20, k=   3, MLE=0.1500, MAP=0.1429, Posterior mean=0.1667
n=  200, k=  30, MLE=0.1500, MAP=0.1490, Posterior mean=0.1524
n= 2000, k= 300, MLE=0.1500, MAP=0.1499, Posterior mean=0.1502

== TASK 8: Softmax regression gradient ==
Probabilities: [0.3333 0.3333 0.3333]
Gradient:
 [[-0.6667  0.3333  0.3333]
 [-1.3333  0.6667  0.6667]]
Cross-entropy loss: 1.0986
Weights after one update:
 [[ 0.0667 -0.0333 -0.0333]
 [ 0.1333 -0.0667 -0.0667]]

== TASK 12: 38 classes, 20% label noise ==
Entropy of 3 classes: 1.585
Entropy of 38 classes: 5.2479
Chance accuracy for 3 classes: 0.3333
Chance accuracy for 38 classes: 0.0263
Noisy-label entropy floor: 1.7638 bits
Accuracy against noisy labels under this noise model: 0.8000
Softmax weights (4096 x 38): 622592 bytes = 0.5938 MiB
Softmax weights with PCA-50: 7752 bytes = 0.0074 MiB
k-NN storage for raw features: 156.25 MiB
k-NN storage for PCA-50 features: 1.9073 MiB

Verification completed successfully.
>>>
```